



第十六章 软件架构重构

王璐璐 wanglulu@seu.edu.cn

廖力 lliao@seu.edu.cn

第16章 软件架构重构

- 16.1 引言
- 16.2 软件重构现状
- 16.3 基于度量的软件架构重构
- 16.4 小结

16.1 引言 (1)

- 软件重构是进行**软件优化**和**提高软件质量**的重要手段之一。
- 在**不改变软件的功能和外部可见性**的情况下，为了**改善软件的结构**，**提高**软件的可读性、可扩展性、可重用性、可维护性等**质量属性**的情况下而对其进行的改造。
- 简而言之，软件重构就是**改进已经写好的软件设计**、**提高软件的某些质量属性**。

16.1 引言 (2)

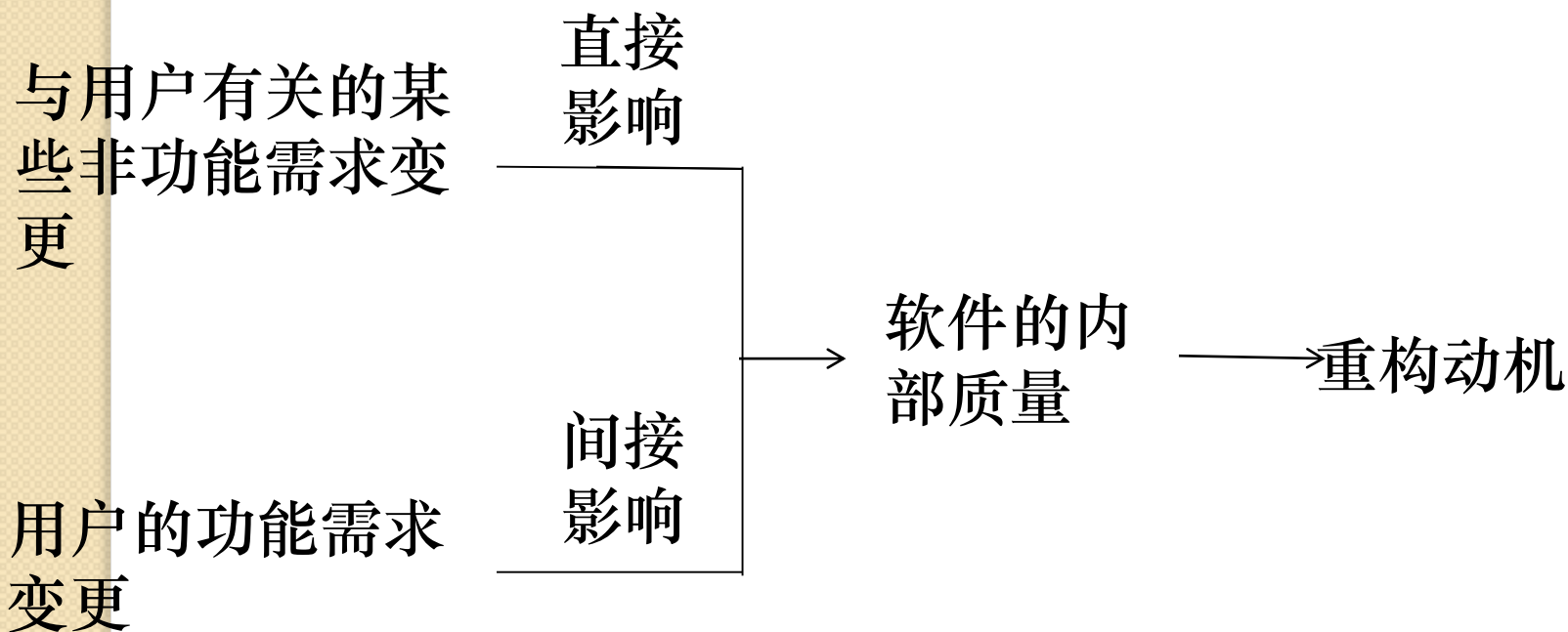
- 软件设计包含软件架构设计、软件详细设计、接口设计、代码设计和算法设计等等，所以软件重构需要考虑的情形有：
 - 软件架构重构 (re-architecture或者architecture reconstruction)
 - 结构调整 (re-structure或者re-organization)
 - 代码重构 (code refactoring) 等

16.1 引言 (3)

重构类别	重构需求	重构目的
架构重构	软件架构级坏味道、设计技术债	通过软件产品整体结构或者组织方式的重构来消除坏味道和技术债，提高软件架构的质量等
代码重构	代码坏味道、代码技术债	通过代码重构来消除坏味道和代码方面的技术债，提高代码质量等

16.1 引言 (4)

- 软件重构背景



16.2 软件重构现状（1）

● 1. 软件重构概念

- 重构（名词）：对软件内部结构的一种调整，目的是在不改变软件可观察行为的前提下，提高其可理解性，降低其修改成本。
- 重构（动词）：使用一系列重构的手法，在不改变软件可观察行为的前提下，调整其结构。
- 重构通过不断地改善软件内部结构来减少软件的复杂性，使得软件更易理解与维护。

16.2 软件重构现状（2）

- I. 软件重构概念
 - 软件重构过程：首先需要进行**重构点识别**与定位，以及**对重构点实施重构操作**，达到重构的目的。

16.2 软件重构现状（2）

- 2.软件重构点识别和定位方法
 - 重构点（refactoring point）指的是需要重构的地方，重构点的识别和定位是实施重构的前提。
 - 根据不同的程序抽象层次、重构粒度以及重构需求，需要利用不同的软件重构点识别和定位方法。

16.2 软件重构现状 (3)

2. 软件重构点识别和定位方法

重构点识别和定位方法分类：

- 基于不定式的识别定位方法
- 基于文本的识别和定位方法
- 基于度量的识别和定位方法
- 基于聚类的识别和定位方法
- 基于设计模式的识别和定位方法

16.2 软件重构现状（4）

- 2.软件重构点识别和定位

（1）**基于不定式**：基于对运行的程序进行动态分析，在特定点(如循环头、程序进入或退出点)动态识别出不变式，然后使用前置条件来确定候选重构点，执行重构后，满足后置条件来确保重构的正确性

（2）**基于文本**：是将源代码经过词法分析，并将得到的 token串作为待检源,再通过文本比对和分析的方法进行检测的。最常见的通过文本进行检测的重构对象是重复代码。其它重构对象还有 switch、平行继承体系、过多的注释等。

16.2 软件重构现状（5）

- 2. 软件重构点识别和定位

（3）**基于度量**：对软件开发项目、过程及其产品进行数据定义、收集以及分析的持续性定量化过程，提供一种客观量化代码特征的方式，可以用来定位重构代码。

（4）**基于聚类**：聚类是指将物理或抽象对象的集合分成由类似的对象组成的多个类的过程。使用聚类分析的方法来定位重构点，主要是针对由于内聚性和耦合性等相关问题引发的一类重构点。

16.2 软件重构现状（6）

- 2. 软件重构点识别和定位

（5）**基于设计模式**：设计模式为重构提供了良好的指导方向。可以利用设计模式来检测常见的重构点。

16.2 软件重构现状（7）

● 3. 重构实施技术

重构实施指的是针对重构点进行实施重构的过程。主要包括如下五类技术：

- 图转换技术（Graph Transformation）
- 程序精化技术（Program Refinement）
- 程序切片技术（Program Slicing）
- 形式概念分析（Formal Concept Analysis）
- 基于模式的重构

16.2 软件重构现状（8）

● 3.软件重构实施技术

（1）图转换技术：重构和图形变换之间有直接对应关系。程序（或其他类型的软件制品）可以表示为图形，重构对应于图形产生规则，重构的应用对应于图形变换。重构前后条件可以表示为应用前和后条件。

（2）程序精化技术：将设计好的程序规约转换成可以逐步执行的程序。

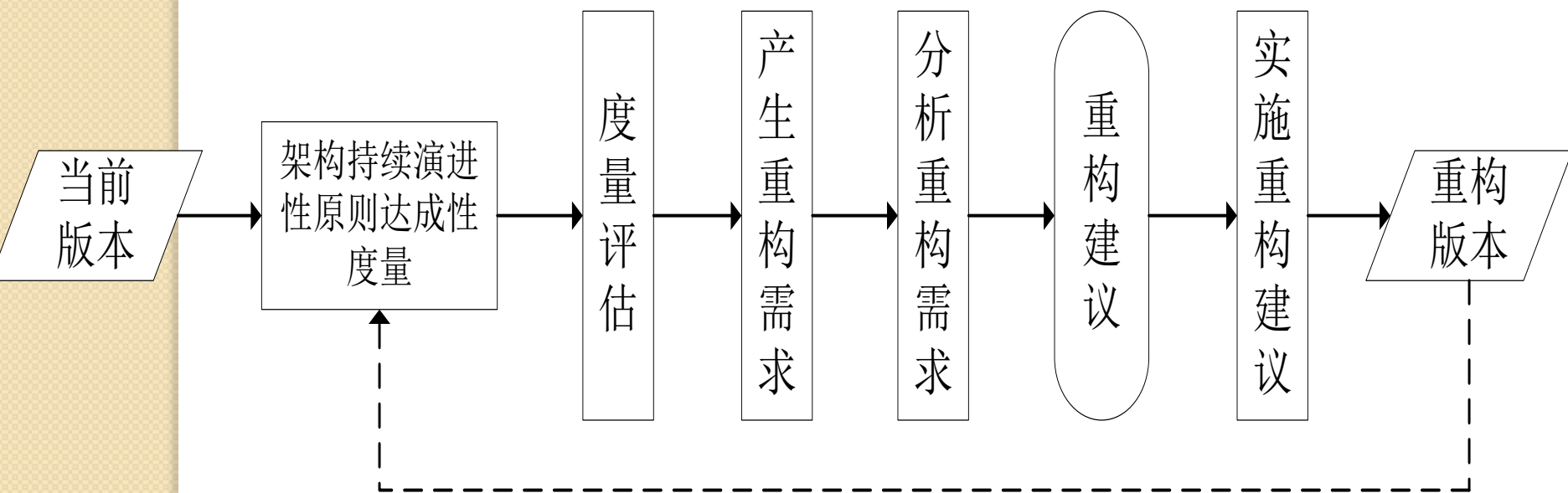
（3）程序切片技术：程序切片是一种提取程序中可能影响某一变量集的所有语句的技术。

（4）形式化概念分析：数据分析提供了一种概念工具。该形式化技术使用格子理论，构造了概念格，其中变量和类成员之间的关系明确，以发现它们的共同属性，并分组。

（5）基于模式的技术：将模式与重构联系起来，模式作为重构的目的，通过实现、趋向和去除模式进行重构。

16.3 基于度量的软件架构重构（1）

该重构发生在一次演进过程结束之后，将软件质量引入重构过程，通过架构度量和评估结果中发现的架构缺陷确定重构对象和重构范围，并提出重构建议。



基于度量评估结果的架构重构过程

16.3基于度量的软件架构重构（2）

- I. 软件架构度量评估

- 选取相关的软件架构持续演进原则，形成达成性度量方法，通过度量评估该软件架构持续演进原则的达成情况。

原则名称	度量参数	度量方法	涉及元素
主体维持原则	N_{change} : 从演进初始版本到演进结果版本的演进过程中组件图中增加、删除或修改的节点数和边数 $N_{unchange}$: 未变化的节点数和边数	$MP = \frac{N_{unchange} - N_{change}}{N_{unchange} + N_{change}}$	组件依赖图中的节点数和边数
平滑演进原则	$Size_{pre}$: 组件在演进初始版本中的规模 $Size_{cur}$: 组件在演进结果版本中的规模 n : 组件的个数 当组件增加或删除时, X_i 取 1。	$SP = 1 - 2 \frac{\sum x_i}{n}$ $x_i = \frac{ Size_{pre} - Size_{cur} }{\max(Size_{pre}, Size_{cur})}$	各组件规模
组件规模最小化原则	$SIZE_{pre}$: 演进初始版本的项目规模 N_{pre} : 演进初始版本的组件数 $SIZE_{cur}$: 演进结果版本的项目规模 N_{cur} : 演进结果版本的组件数	$CP = \frac{\frac{SIZE_{pre}}{N_{pre}} - \frac{SIZE_{cur}}{N_{cur}}}{\frac{SIZE_{pre}}{N_{pre}} + \frac{SIZE_{cur}}{N_{cur}}}$	系统规模、组件个数
模块独立演进原则	N_{pre} : 演进初始版本组件图中的边数 n_{pre} : 演进初始版本组件图中的节点数 N_{cur} : 演进结果版本组件图中的边数 n_{cur} : 演进结果版本组件图中的节点数	$IP = \frac{\frac{N_{pre}}{n_{pre}} - \frac{N_{cur}}{n_{cur}}}{\frac{N_{pre}}{n_{pre}} + \frac{N_{cur}}{n_{cur}}}$	组件依赖图中的节点数和边数
外部接口稳定原则	N_{change} : 从演进初始版本到演进结果版本的演进过程中增加、删除或修改的外部接口数 $N_{unchange}$: 未变化的外部接口数	$EP = \frac{N_{unchange} - N_{change}}{N_{unchange} + N_{change}}$	外部接口

16.3 基于度量的软件架构重构（4）

• 1. 软件架构度量评估

- 通过这些度量就可以计算得出目前相邻版本之间的演化原则的达成度。
- 取值范围都是 $[-1, 1]$ ，当指标度量结果大于0时，表示从演进初始版本到演进结果版本的演进过程达成相应的持续演进原则子指标，且度量结果越大时表示达成效果越好；
- 当度量结果小于0时，表示从演进初始版本到演进结果版本的演进过程未达成相应的持续演进原则子指标，且度量结果越小时表示达成效果越差。

16.3基于度量的软件架构重构（5）

- 2. 产生重构需求

架构持续演进原则达成性度量结果作为识别重构需求的入口，当度量过程结束后，识别度量结果中未达成的架构持续演进原则子指标，通过对子指标度量公式的分析判断是否可以通过重构操作达成子指标。

16.3 基于度量的软件架构重构 (6)

• 3. 分析重构需求

各子指标度量参数的来源有较大的差异，主要来源于架构层次和代码层次两个层次，有些子指标的度量参数同时来源于两个层次：

层次	相关指标	相关元素
组件层次	主体维持原则、组件规模最小化原则、模块独立演进原则	组件出入度、组件数量
代码层次	平滑演进原则、组件规模最小化原则、外部接口稳定性原则、复杂性可控原则	项目规模、圈复杂度、外部接口

16.3 基于度量的软件架构重构（7）

• 3. 分析重构需求

度量原则	重构背景	重构主要原因
主体维持原则	(1) 组件依赖图中的组件没有发生变动，组件之间的依赖关系发生了较大变化； (2) 组件依赖图中的组件发生了变化，从而导致组件之间的依赖关系也发生了变化。	组件和组件之间的依赖关系变动太大
平滑演进原则	(1) 组件本身规模发生了较大的变动； (2) 组件增加或删除。	组件规模平均变化率过大
组件规模最小化原则	(1) 软件规模不变，组件数降低； (2) 组件数不变，软件规模增大； (3) 软件规模减少，组件数减少，且软件规模的减少比例小于组件数减少的比例； (4) 软件规模增大，组件数增大，且软件规模的增加比例大于组件数增大的比例	组件平均规模过大

16.3 基于度量的软件架构重构 (8)

• 3. 分析重构需求

度量原则	重构背景	重构主要原因
模块独立 演进原则	(1) 组件依赖图中节点数不变, 边数增多 (2) 组件依赖图中边数不变, 节点数减少 (3) 组件依赖图中节点数增多, 边数增多, 且边数的增加比例大于节点数增加的比例 (4) 组件依赖图中节点数减少, 边数减少, 且边数减少的比例小于节点数减少的比例	组件依赖图中节点数和边数比值过大, 即组件之间依赖关系过于复杂
外部接口 稳定原则		外部接口变动太大
复杂性可 控原则	(1) 软件规模不变, 圈复杂度增大; (2) 圈复杂度不变, 软件规模减小; (3) 软件规模增大, 圈复杂度增大, 且圈复杂度增大的比例大于软件规模增大的比例; (4) 软件规模减小, 圈复杂度减小, 且圈复杂度减小的比例小于软件规模减少的比例。	圈复杂度平均密度过大

16.3 基于度量的软件架构重构（9）

- 4. 建议重构操作

在明确重构需求以及明确架构缺陷之后，我们应针对不同重构需求下的不同架构缺陷选择恰当的方法对系统进行重构，重构方法应明确重构对象以及重构操作。

通过前文对架构持续演进原则度量结果的评估，涉及到组件层次的相关指标，重构操作主要在组件依赖图上对组件及组件之间的依赖关系进行修改，可能的操作有拆分组件、提取组件等；涉及到代码层次的相关指标，重构操作需要对代码进行修改，可能的操作有简化代码逻辑结构、消除冗余代码等。

16.3 基于度量的软件架构重构（10）

• 4. 建议重构操作

架构持续演进原则子指标	重构建议
组件规模最小化原则	拆分组件或提取组件
模块独立演进原则	降低文件之间耦合
复杂性可控原则	降低文件圈复杂度
主体维持原则	定位组件具体变化反馈用户
平滑演进原则	定位规模具体变化反馈用户
外部接口稳定原则	定位接口具体变化反馈用户

16.3 基于度量的软件架构重构（11）

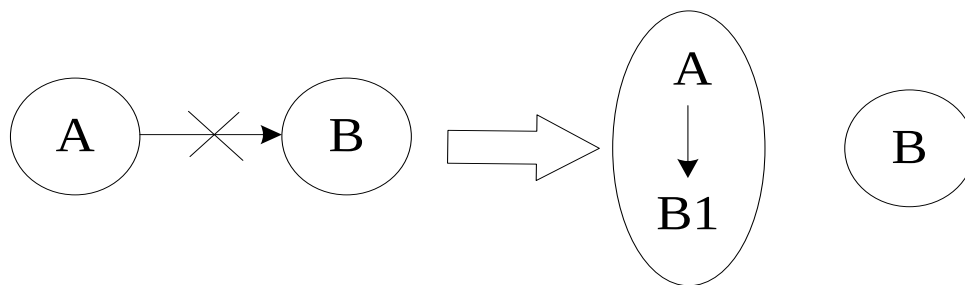
- **5. 实施重构操作**
- 重构建议的实施部分需要架构师和软件设计人员根据架构和代码的具体情况进行操作
- 实施策略：
 - **（1）拆分或提取组件**
 - **（2）降低文件之间耦合**
 - **（3）降低圈复杂度**

16.3 基于度量的软件架构重构（12）

- **5. 实施重构操作**
- **（1）拆分或提取组件：**将组件中内聚性较强的部分拆分或者提取出来形成一个新的组件。拆分组件通过对组件中规模最大或文件数量最多的组件进行拆分，提取组件的操作依靠的是组件内部依赖关系和组件之间依赖关系的分析。

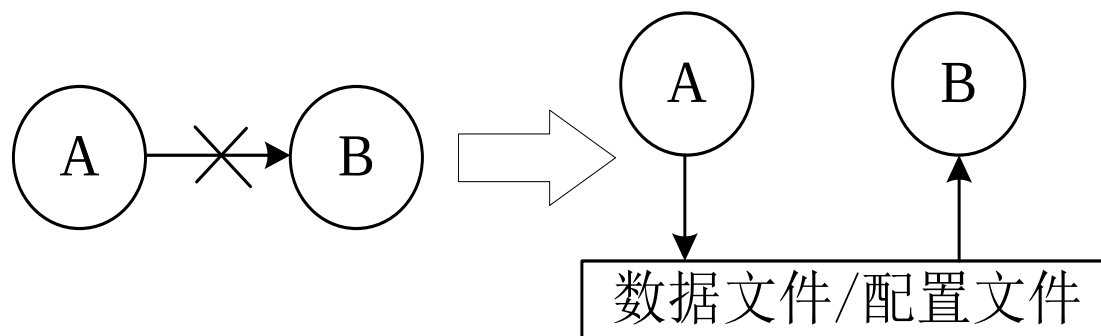
16.3 基于度量的软件架构重构 (13)

- 5. 实施重构操作
- (2) **降低文件之间耦合**：文件之间的依赖关系主要有方法调用、实例化、继承、参数依赖等，删减文件之间的依赖主要可以通过以下几种方式来实现：
 - **桩替换**：桩代码是指用来代替关联代码或者未实现代码的代码



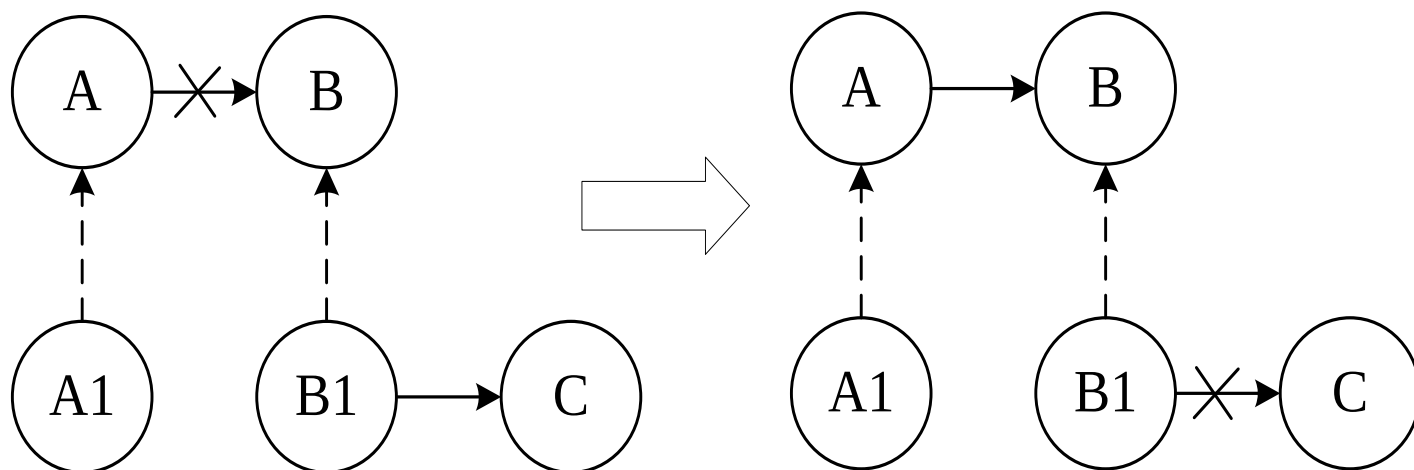
16.3 基于度量的软件架构重构 (14)

- 5. 实施重构操作
- (2) 降低文件之间耦合：
 - **数据依赖传递**：通过设置数据文件或者配置文件来转移文件A和文件B之间的依赖关系，将文件A和文件B转换为相对独立的两个文件。



16.3 基于度量的软件架构重构 (15)

- 5. 实施重构操作
- (2) 降低文件之间耦合:
 - 结构调整:

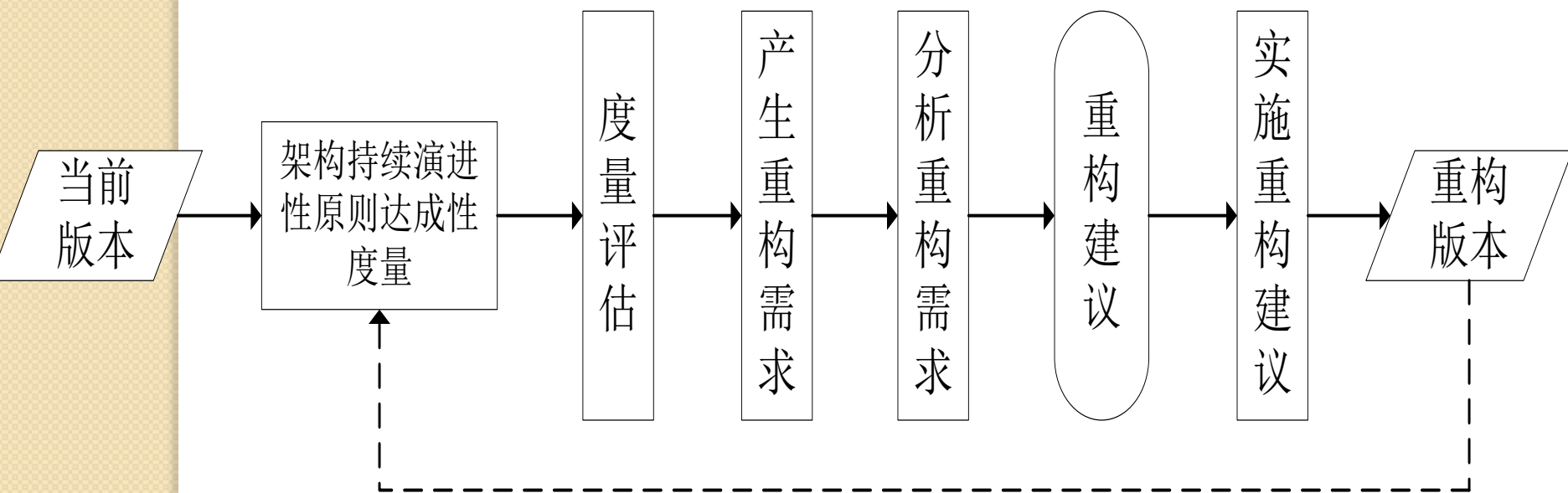


16.3基于度量的软件架构重构（16）

- 5. 实施重构操作
- （3）降低圈复杂度：圈复杂度在数量上表现为基本路径条数，圈复杂度大则程序代码的判断逻辑复杂。
- 降低圈复杂度的方法主要分为三类：
 - 重新组织函数：包括提炼函数和替换算法。提炼函数主要是指将一大段代码中可以被组织在一起并独立出来的一段代码放进一个独立的函数中；替换算法则是指将某个算法替换为另一个更清晰的算法。
 - 简化条件表达式：分解条件式、合并条件式、合并重复的条件片段等。
 - 简化函数调用：将查询函数和修改函数分离、为方法增加参数、分解函数功能等方法来达到目的。

16.3 基于度量的软件架构重构（17）

重构实施完成后，即可再次进行基于度量的软件架构评估，以度量重构效果。



基于度量评估结果的架构重构过程

小结

本章讨论了：

- 软件架构重构的基本概念、基本原理和基本方法
- 软件架构重构的动机和目的
- 如何利用软件架构持续演进原则达成性度量和评估进行软件架构重构的方法。